SOFTWARE REQUIREMENT SPECIFICATION

Online Medical Consultation Platform

Supervisor:

Majid Khawar



UNIVERSITY OF SOUTHERN PUNJAB

Department of Computer Science & IT

Group Member 1	Muhammad Danial
Registration No.	BSCSW-2.5Y-021R24-4
Group Member 2	Muhammad Bilal
Registration No.	BSCSW-2.5Y-021R24-1
Group Member 3	Mudassar Javed
Registration No.	BSCSW-2.5Y-021R24-5

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Problem Statement 4
1.4 Background and Motivation 5
1.5 Project Objectives 5
1.6 Definitions, Acronyms, and Abbreviations 6
1.7 References 7
1.8 Document Overview 7
2. Overall Description 8
2.1 Background and Product Perspective 8
2.2 Existing Systems in the Market 8
2.2.1 Practo 8
2.2.2 Zocdoc 9
2.2.3 Marham 9
2.2.4 Comparative Analysis 9
2.3 Product Functions 10
2.4 User Classes and Characteristics 10
2.4.1 Patient 11
2.4.2 Doctor 11
2.4.3 Administrator 11
2.5 Operating Environment 11
2.6 Design and Implementation Constraints 11
2.7 Assumptions and Dependencies 12
2.8 Scope Exclusions 12
2.9 Limitations 12
3. System Features 13
3.1 User Authentication and Session Management 13
3.2 AI-Driven NLP Symptom Triage Engine 13
3.3 Doctor Directory and Search 14
3.4 Appointment Scheduling 14
3.5 Secure Messaging 14
3.6 Digital Prescription Management 15

3.7 Billing and Invoice Generation 15
3.8 Administrator Control Panel 16
3.9 Profile Management 16
4. External Interface Requirements 17
4.1 User Interface 17
4.2 Hardware Interface 17
4.3 Software Interface 17
4.4 Communication Interface 17
4.5 Database Requirements 17
5. UML and System Diagrams 18
5.1 Context Diagram (DFD Level 0) 18
5.2 System Architecture Diagram 18
5.3 Use Case Diagram 19
5.4 Sequence Diagram — AI NLP Symptom Triage 19
5.5 Sequence Diagram — Appointment Booking 20
5.6 Sequence Diagram — Secure Messaging 21
5.7 Activity Diagram — Consultation Workflow 21
5.8 Entity Relationship Diagram (ERD) 22
6. Non-Functional Requirements 23
7. Conclusion 24
Appendix A: Glossary 25
Appendix B: Bibliography and References 27
B.1 Standards and Guidelines 27
B.2 Technical Documentation 27
B.3 Healthcare and Telemedicine References 27
B.4 AI and NLP in Healthcare 27
B.5 Software Engineering References 27
Appendix C: Document Revision History 28

1. Introduction

1.1 Purpose

This document is the Software Requirement Specification (SRS) for the Online Medical Consultation Platform, developed as a Final Year Project for the Department of Computer Science and Information Technology at the University of Southern Punjab, Multan. It formally defines the functional and non-functional requirements of the proposed system and serves as the primary reference for the design, development, and evaluation of the platform.

The document is addressed to the project supervisor, the academic evaluation committee, and the development team. It provides a comprehensive, unambiguous description of the system's capabilities, interfaces, constraints, and quality attributes, ensuring that all stakeholders share a common understanding of what the system will do and how it will behave.

1.2 Scope

The Online Medical Consultation Platform is a web-based healthcare application that facilitates remote, outpatient consultations between verified medical practitioners and registered patients. The system addresses a well-established accessibility problem: patients in underserved or remote areas frequently struggle to reach the correct medical specialist in a timely manner. By introducing an AI-driven, Python-based Natural Language Processing (NLP) Symptom Triage Engine, the platform enables patients to describe their medical concerns in free-text and be automatically directed to the most appropriate specialist without requiring any prior medical knowledge.

Beyond triage, the platform provides a complete consultation workflow. Patients can browse a verified directory of doctors, schedule appointments with conflict detection, communicate securely through an asynchronous messaging system, and subsequently view digital prescriptions and billing invoices. Doctors manage their professional profiles, consultation schedules, and patient records from a dedicated interface. An administrator maintains overall platform integrity through a control panel covering user verification, account management, and system monitoring.

The application is implemented using a micro-architecture integrating HTML5, CSS3, JavaScript, PHP, MySQL, and a Python NLP module, designed to operate within a standard XAMPP server environment.

1.3 Problem Statement

Access to qualified healthcare remains a critical challenge in Pakistan. According to the World Health Organization, Pakistan has approximately 1 doctor per 1,000 patients — significantly below the recommended ratio of 1:500. Rural populations, which constitute nearly 63% of the country, face even greater barriers, including geographical distance to medical facilities, lack of transportation, and unavailability of specialist doctors outside major cities such as Lahore, Karachi, and Islamabad.

In the existing healthcare model, patients must physically visit clinics or hospitals to consult a doctor. This process is time-consuming, costly, and inaccessible for working individuals, the elderly, and those in remote regions. Furthermore, patients without medical knowledge often do not know which type of specialist they need, leading to multiple wasted visits and delayed treatment.

While telemedicine platforms exist globally, no locally developed, openly deployable solution tailored to the Pakistani context integrates AI-based symptom triage, appointment management, secure communication, and digital prescriptions within a single unified platform. This project directly addresses these gaps by building the Online Medical Consultation Platform.

1.4 Background and Motivation

The emergence of telemedicine as a viable alternative to traditional in-person healthcare has been accelerated by advancements in internet connectivity. In Pakistan, the PTA reported internet penetration reached 54% by 2024, creating a viable technical foundation for web-based healthcare services. Commercial platforms such as Marham, Sehat.com.pk, and Oladoc exist; however, these are proprietary systems that rely on manual specialty selection rather than intelligent, AI-driven patient routing.

The motivation for this project arises from three observations: no open-source, locally deployable medical consultation platform is available for small clinics or academic institutions in Pakistan; existing platforms lack an AI-based triage mechanism; and the availability of medical datasets and Python NLP libraries makes it academically feasible to train a Machine Learning classifier for specialty prediction. This project fills that gap by delivering an academically transparent, AI-enhanced system.

1.5 Project Objectives

The specific objectives of this project are as follows:

- To design and implement a web-based telemedicine platform enabling remote outpatient consultations between verified doctors and registered patients.

- To develop an AI-Driven NLP Symptom Triage Engine using Python (NLTK/spaCy) that processes free-text patient input, extracts medical keywords, and predicts the appropriate medical specialty using a Machine Learning model.
- To implement a secure, asynchronous messaging system linking patients and doctors within confirmed appointments.
- To provide a complete consultation lifecycle encompassing appointment scheduling, digital prescriptions, and billing invoice generation.
- To enforce role-based access control across three user classes — Patient, Doctor, and Administrator.
- To deploy the system on a standard LAMP/XAMPP stack, accessible to small institutions without requiring cloud infrastructure.
- To produce a fully documented, maintainable codebase adhering to standard software engineering practices.

1.6 Definitions, Acronyms, and Abbreviations

The following terms are used consistently throughout this document:

Term	Definition
NLP	Natural Language Processing. A branch of AI enabling computers to understand human language.
Machine Learning (ML)	A subset of AI where systems learn patterns from data to make predictions.
TF-IDF	Term Frequency-Inverse Document Frequency. An NLP algorithm for keyword extraction.
SRS	Software Requirement Specification – this document.
RBAC	Role-Based Access Control. Restricts system access based on a user's assigned role.
Triage	The process of routing patients based on symptoms to the appropriate medical specialty.
AJAX	Asynchronous JavaScript and XML. Background browser-server communication without full page reload.
PDO	PHP Data Objects. A database abstraction layer for parameterised queries.
API	Application Programming Interface. Defines how modules exchange data.
XAMPP	A cross-platform server stack comprising Apache, MySQL, and PHP.
DFD	Data Flow Diagram. A graphical representation of data flows through the system.
ERD	Entity Relationship Diagram. Models the database structure and relationships.
Patient	A registered end user seeking medical consultation through the platform.
Doctor	A verified healthcare practitioner registered on the platform.
Administrator	The system operator responsible for user verification, platform integrity, and monitoring.

1.7 References

The following sources and standards informed the preparation of this document:

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering: Requirements Engineering.
- Project Scope Document — Online Medical Consultation Platform (Department of CS&IT, USP, 2026).
- World Health Organization. (2023). Pakistan Health Profile. WHO Global Health Observatory.
- Pakistan Telecommunication Authority. (2024). Annual Report on Internet Usage in Pakistan.
- PHP Manual — The PHP Group (https://www.php.net/manual).
- Python Software Foundation. (2024). Python 3 Reference Manual.

1.8 Document Overview

This specification is organized into seven main sections plus three appendices. Section 1: introduction, scope, problem statement, background, objectives, and terminology. Section 2: overall description including existing systems, user classes, and operating environment. Section 3: system features and functional requirements. Section 4: external interface requirements. Section 5: UML and system diagrams. Section 6: non-functional requirements. Section 7: conclusion. Appendices A–C contain the glossary, bibliography, and revision history.

2. Overall Description

2.1 Background and Product Perspective

The Online Medical Consultation Platform operates as an independent, self-contained web application organized around a micro-architecture integrating two primary technology stacks: PHP and Python. The presentation layer consists of HTML, CSS, and JavaScript. The application layer comprises PHP scripts for business logic and session management, and a Python NLP micro-module for AI-driven symptom triage. The data layer is a MySQL relational database. PHP communicates with the Python NLP module via a lightweight internal REST API (Flask), providing seamless AI integration and making the system independently deployable on any standard hosting environment. Virtual meeting sessions use external services such as Google Meet, for which the system stores and presents a shareable meeting link.

2.2 Existing Systems in the Market

Before designing the Online Medical Consultation Platform, a survey of existing telemedicine and doctor appointment systems was conducted. Three representative systems were selected: Practo (international), Zocdoc (US-based), and Marham (Pakistan-specific).

2.2.1 Practo

Practo is one of the largest online healthcare platforms in South Asia. It allows patients to search for doctors by specialty, book in-clinic and online appointments, and access digital prescriptions. However, Practo does not include an AI-driven symptom triage engine — patients must manually select their specialty. The platform is proprietary, cloud-hosted, and unavailable for academic deployment.

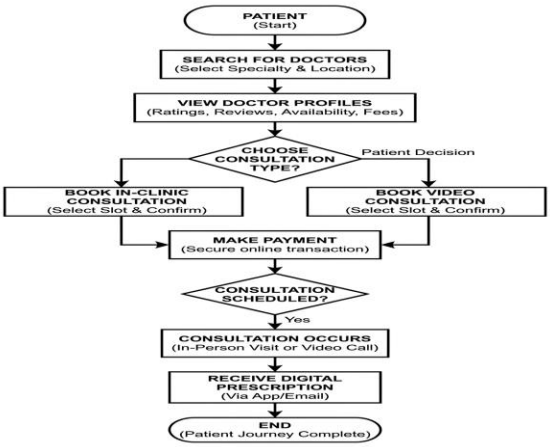


Figure 2.1 — Practo Operational Workflow Diagram

2.2.2 Zocdoc

Zocdoc is a US-based healthcare scheduling platform enabling patients to find doctors by specialty, check real-time availability, and book appointments. Its key strength is insurance verification. However, Zocdoc is geo-restricted to the United States and does not offer an AI-based symptom triage system.

Figure 1: Zocdoc Platform Working Model Flowchart

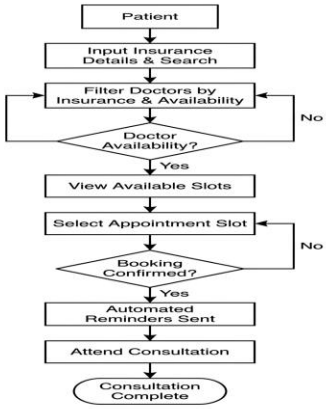


Figure 2.2 — Zocdoc Operational Workflow Diagram

2.2.3 Marham

Marham is Pakistan's leading online doctor consultation and appointment booking platform. It lists thousands of verified doctors with consultation fees and availability. While Marham serves the Pakistani market, it is a commercial platform. It requires patients to manually select a specialty — no AI or NLP triage is offered. It is not available for self-hosted academic deployment.

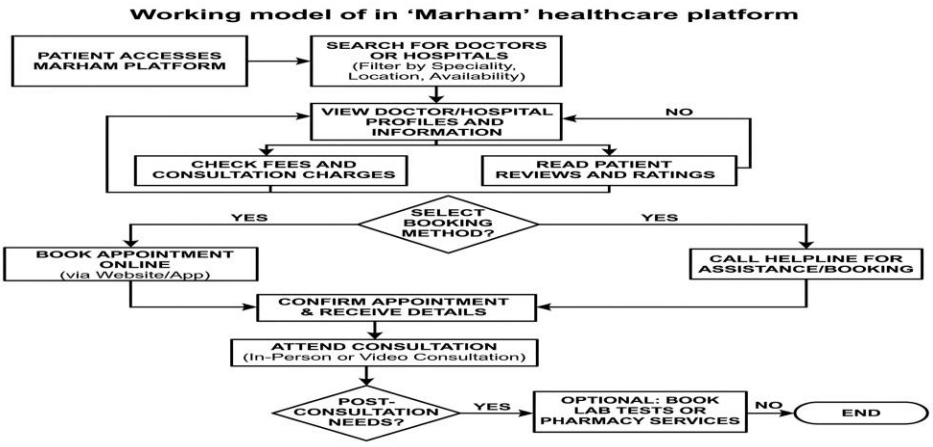


Figure 2.3 — Marham Operational Workflow Diagram

2.2.4 Comparative Analysis

The table below compares the key features of existing systems against the proposed Online Medical Consultation Platform:

Feature	Practo	Zocdoc	Marham	Our System
AI NLP Free-Text Triage	No	No	No	**Yes**
ML-Based Specialty Prediction	No	No	No	**Yes**
Available in Pakistan	No	No	Yes	**Yes**
Appointment Scheduling	Yes	Yes	Yes	**Yes**
Secure Doctor–Patient Messaging	Yes	Yes	Partial	**Yes**
Digital Prescriptions	Yes	Yes	Yes	**Yes**
Invoice / Billing	Yes	Yes	Yes	**Yes**
Admin Verification Panel	Yes	Yes	Yes	**Yes**
Open / Self-hosted Deployment	No	No	No	**Yes**
Free for Academic Use	No	No	No	**Yes**

The comparison confirms that no existing system simultaneously offers AI-based symptom triage, availability in Pakistan, self-hosted open deployment, and a complete consultation workflow. The proposed system uniquely satisfies all requirements, establishing clear academic and practical significance.

2.3 Product Functions

At a high level, the platform provides the following core capabilities:

- User registration, authentication, and role-based session management for Patients, Doctors, and Administrators.
- AI NLP Symptom Triage: a Python-based engine that processes free-text patient input, extracts medical keywords, and uses a trained Machine Learning model to return a list of matching verified doctors.
- Doctor Directory: a browsable, searchable listing of verified medical practitioners.
- Appointment Scheduling: a request and approval workflow with conflict detection.
- Secure Messaging: an asynchronous text channel between Patient and Doctor via AJAX polling.
- Digital Prescription Management: structured prescription records linked to completed consultations.
- Billing and Invoice Generation: doctor-generated invoices for completed consultations.
- Administrator Control Panel: doctor verification, user account management, and audit log review.
- Profile Management: self-service updates to personal and professional information.

2.4 User Classes and Characteristics

The platform serves three distinct user roles, each with a separate interface and defined permissions enforced through role-based access control.

2.4.1 Patient

Patients are individuals seeking remote medical consultation. They are assumed to have basic familiarity with web browsing but no medical or technical expertise. Upon registration, a patient gains access to the AI triage tool, doctor directory, appointment scheduler, messaging interface, and prescription and invoice viewer.

2.4.2 Doctor

Doctors are verified healthcare practitioners who have provided confirmed professional credentials. They manage availability, respond to appointment requests, communicate with patients via the secure messaging system, issue digital prescriptions, and generate invoices. A doctor's specialty can only be changed by an Administrator.

2.4.3 Administrator

The Administrator maintains platform integrity by reviewing and approving or rejecting doctor registration applications, managing user accounts, and monitoring the audit log. Administrators do not conduct consultations and do not have access to private messages between Patients and Doctors.

2.5 Operating Environment

The system is a web application accessible through any modern desktop or tablet web browser (Chrome, Firefox, Edge, Safari). The backend requires a PHP-enabled web server; the deployment environment is XAMPP, bundling Apache 2.4, PHP 8.x, and MySQL 8.0. The AI module requires Python 3.x with scikit-learn, NLTK/spaCy, and pandas installed on the same server. Minimum recommended screen resolution is 1024 × 768 pixels.

2.6 Design and Implementation Constraints

- The messaging system uses AJAX polling rather than WebSockets. This accepts a slight latency of approximately three seconds on message delivery.
- Video consultations are facilitated through external meeting services. The platform stores only the meeting link; no audio or video is processed internally.
- No live payment gateway is integrated. Billing is handled through invoices generated within the system; financial settlement occurs offline.
- The AI module runs as a Python Flask micro-service and is invoked by PHP via an internal HTTP REST API call, enabling independent deployment and scalability of each component.
- The system is designed for standard outpatient consultations and is not intended for emergency triage or life-critical decision support.

2.7 Assumptions and Dependencies

- All users possess access to a device with a modern web browser and a stable internet connection.
- Doctors will submit accurate professional credentials during registration.
- The Kaggle dataset used for ML model training accurately represents real-world symptom-to-disease correlations.
- Python 3.x with required libraries is installed and accessible from the web server process.
- External meeting services such as Google Meet will remain available for virtual sessions.
- The XAMPP/LAMP environment will be correctly configured for the application's operation.

2.8 Scope Exclusions

The following capabilities are explicitly outside the system boundary and will not be implemented:

- Proprietary real-time video or audio streaming within the platform.
- Integration with external Hospital Management Systems (HMS) or EHR systems.
- Native iOS or Android mobile application development.
- Live online payment processing or payment gateway integration.

2.9 Limitations

System performance depends on server hardware and a stable internet connection. The platform supports standard outpatient consultations only. The AI triage engine provides a specialty recommendation based on symptom text; it does not provide a diagnosis and must not be used as a substitute for professional medical judgment.

3. System Features

3.1 User Authentication and Session Management

This module provides the entry point to the system and controls access to all role-protected resources. Upon successful login, the system creates an authenticated session associated with the user's role and redirects them to the appropriate dashboard. All subsequent requests to protected pages are validated against the active session.

Req. ID	Functional Requirement
FR-01	The system shall allow users to register with a full name, email address, password, and role selection.
FR-02	The system shall authenticate users via email and password and create a role-bound session upon success.
FR-03	The system shall redirect authenticated users to the dashboard corresponding to their assigned role.
FR-04	The system shall deny access to any protected page for unauthenticated users and redirect them to the login page.
FR-05	The system shall allow users to log out and invalidate their current session.

3.2 AI-Driven NLP Symptom Triage Engine

The AI Symptom Triage Engine is the distinguishing feature of this platform. Rather than requiring patients to know which type of doctor they need, patients type a free-text description of their symptoms in natural language. A Python script processes this input using NLP techniques — tokenisation, stopword removal, and keyword extraction — and evaluates the result against a Machine Learning model trained on a medical dataset (from Kaggle). The model predicts the most appropriate medical specialty. PHP retrieves and returns a shortlist of verified doctors in that specialty. If the confidence score falls below a defined threshold, the system gracefully falls back to displaying the full doctor directory.

Req. ID	Functional Requirement
FR-06	The system shall provide a free-text input field for the patient to describe symptoms in natural language.
FR-07	The system shall pass the sanitised input from PHP to the Python NLP module via an internal REST API HTTP request.
FR-08	The Python module shall apply NLP preprocessing (tokenisation, stopword removal, lemmatisation) to the input text.
FR-09	The Python module shall transform processed keywords into a TF-IDF feature vector and apply the trained ML classifier to predict the best-matching medical specialty.
FR-10	The system shall retrieve and return a list of verified doctors registered under the predicted specialty.

FR-11	The system shall display an informative fallback message and link to the doctor directory if the AI confidence is below the threshold or the script fails.

3.3 Doctor Directory and Search

The doctor directory serves as the platform's public-facing practitioner index. Verified doctors appear on the landing page and within triage result sets. Each listing displays the doctor's full name, medical specialty, and professional biography. Patients may filter the directory by specialty without using the triage workflow.

Req. ID	Functional Requirement
FR-12	The system shall display all verified doctors on the public landing page with name, specialty, and biography.
FR-13	The system shall support filtering of the doctor directory by medical specialty.
FR-14	The system shall present doctor listings within triage results filtered to the predicted specialty.

3.4 Appointment Scheduling

The appointment module enables patients to formally request consultations by choosing a preferred date and time. The system validates the request against the doctor's existing appointments to detect and prevent scheduling conflicts. Approved appointments transition from pending to confirmed, at which point the doctor may attach an external meeting link.

Req. ID	Functional Requirement
FR-15	The system shall allow a patient to submit an appointment request specifying doctor, preferred date, and time.
FR-16	The system shall check the requested date and time against the doctor's existing confirmed appointments to detect conflicts.
FR-17	The system shall reject conflicting requests and prompt the patient to choose an alternative time.
FR-18	The system shall assign a status of Pending to all newly submitted appointment requests.
FR-19	The system shall notify the doctor of a new pending appointment request.
FR-20	The system shall allow the doctor to accept or decline any pending appointment request.
FR-21	The system shall update appointment status to Confirmed, Completed, or Cancelled as appropriate.
FR-22	The system shall allow the doctor to attach an external meeting link to a confirmed appointment.

3.5 Secure Messaging

The messaging module provides an asynchronous text-based communication channel linking a patient and a doctor within a specific appointment context. Messages are stored persistently and displayed in chronological order. The interface uses AJAX polling at three-second intervals to retrieve new messages without requiring a full page refresh.

Req. ID	Functional Requirement
FR-23	The system shall allow authenticated patients and doctors to exchange text messages within an appointment context.
FR-24	The system shall store each message with sender ID, timestamp, appointment reference, and message body.
FR-25	The system shall retrieve new messages via AJAX polling at intervals not exceeding three seconds.
FR-26	The system shall display all messages in chronological order with sender name and timestamp.

3.6 Digital Prescription Management

Upon completion of a consultation, the attending doctor may issue a formal digital prescription linked to the appointment record. The prescription records prescribed medications, dosage instructions, and clinical notes. Records are stored securely and permanently accessible to the patient through their prescription history.

Req. ID	Functional Requirement
FR-27	The system shall allow a doctor to issue a digital prescription for any appointment marked as Completed.
FR-28	The system shall store the prescription with doctor ID, appointment reference, issue date, and prescription body.
FR-29	The system shall allow patients to view all prescriptions issued to them through their profile history.

3.7 Billing and Invoice Generation

Doctors may generate a billing invoice at the conclusion of a consultation. The invoice records the consultation fee, generation date, and payment status. Since the platform does not integrate a live payment gateway, settlement occurs offline. The invoice record remains available within the system for reference by both parties.

Req. ID	Functional Requirement
FR-30	The system shall allow a doctor to generate a billing invoice for a completed consultation.
FR-31	The system shall record the invoice amount, generation date, and payment status.

FR-32	The system shall allow patients to view all invoices associated with their consultations.

3.8 Administrator Control Panel

The administrator module provides tools to maintain platform trustworthiness and operational integrity. The primary responsibility is reviewing incoming doctor registration applications and approving or rejecting them based on provided credentials. Administrators also manage user accounts and review the audit log.

Req. ID	Functional Requirement
FR-33	The system shall allow the Administrator to review, approve, or reject pending doctor registration applications.
FR-34	The system shall allow the Administrator to suspend, activate, or permanently delete user accounts.
FR-35	The system shall maintain an audit log recording all critical actions including verifications, deletions, and configuration changes.
FR-36	The system shall present platform metrics on the administrator dashboard, including total users, active appointments, and unverified doctor applications.

3.9 Profile Management

All registered users may update their basic account information from the profile settings page. Doctors may additionally update their professional biography. Medical specialty assignments and verification status are restricted to administrator-only changes.

Req. ID	Functional Requirement
FR-37	The system shall allow all users to update their full name and email address from the profile settings page.
FR-38	The system shall allow all users to change their password, subject to confirmation of the current password.
FR-39	The system shall allow doctors to update their professional biography.
FR-40	The system shall restrict changes to a doctor's medical specialty to Administrator-initiated actions only.

4. External Interface Requirements

4.1 User Interface

The platform provides a consistent, responsive interface across all user roles. The landing page presents the platform's value proposition, the verified doctor directory, and a call-to-action for registration. Each authenticated role accesses a tailored dashboard: the patient dashboard highlights the AI symptom triage tool, upcoming appointments, and recent messages; the doctor dashboard presents pending appointment requests, messaging, and prescription tools; the administrator dashboard summarises platform statistics and pending verification requests.

4.2 Hardware Interface

The system is accessed through standard personal computers, laptops, or tablet devices. No specialized peripheral hardware is required. A stable internet connection is necessary. Minimum screen resolution of 1024 × 768 pixels is recommended.

4.3 Software Interface

The backend requires Apache 2.4+ with PHP 8.x, as provided by XAMPP. Data persistence uses MySQL 8.0 via PHP PDO with prepared statements. The AI module requires Python 3.x with scikit-learn, NLTK/spaCy, and pandas. No browser plugins are required on the client side.

4.4 Communication Interface

Standard HTTP/HTTPS protocols govern all browser-server communication. Form submissions follow the standard HTTP request-response model. The triage module and messaging system exchange data in JSON format through PHP API endpoints. Message synchronization uses AJAX polling at three-second intervals.

4.5 Database Requirements

The system requires a relational database with the following primary entities:

- users — stores credentials, role, verification status, and profile data for all user types.
- appointments — records requests with foreign keys to patient and doctor, date, time, status, and meeting link.
- messages — stores individual messages with sender ID, appointment reference, message body, and timestamp.
- prescriptions — records prescription details linked to a completed appointment and issuing doctor.
- invoices — stores invoice amount, generation date, payment status, and appointment reference.
- audit_log — records critical system events with user ID, action type, timestamp, and description.

5. UML and System Diagrams

This section presents the key structural and behavioural diagrams modelling the Online Medical Consultation Platform, including a Context Diagram, System Architecture Diagram, Use Case Diagram, three Sequence Diagrams, an Activity Diagram, and an Entity Relationship Diagram.

5.1 Context Diagram (DFD Level 0)

The Context Diagram defines the complete system boundary and illustrates how the platform interacts with its external entities: Patient, Doctor, and Administrator. Incoming data flows represent inputs (credentials, symptom descriptions, appointment requests, messages, verification decisions); outgoing flows represent outputs (recommendations, appointment status, prescriptions, invoices, and audit logs).

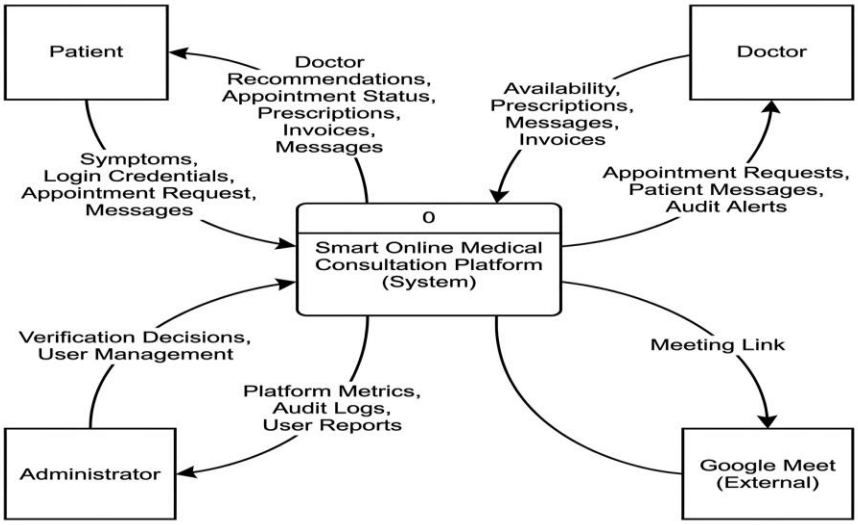


Figure 5.1 — Context Diagram (DFD Level 0): System Boundary and External Entities

5.2 System Architecture Diagram

The System Architecture Diagram presents the three-tier structure of the platform. The Presentation Layer contains all client-side components. The Application Layer contains the PHP business logic and the Python NLP micro-module, connected through a shell execution bridge. The Data Layer contains the MySQL database. This diagram highlights the key design decision of integrating the AI module into the same server environment as the PHP application.

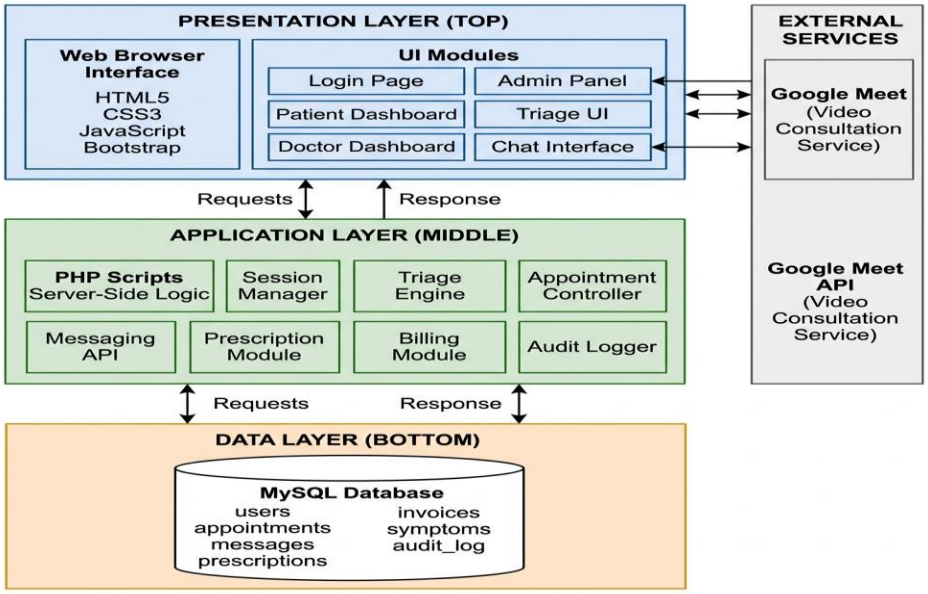


Figure 5.2 — System Architecture Diagram: Three-Tier Architecture with Python AI Module

5.3 Use Case Diagram

The Use Case Diagram identifies the three primary actors — Patient, Doctor, and Administrator — alongside the major use cases available to each role. The AI Triage use case includes an invoke sub-case representing the Python NLP module execution and ML model evaluation.

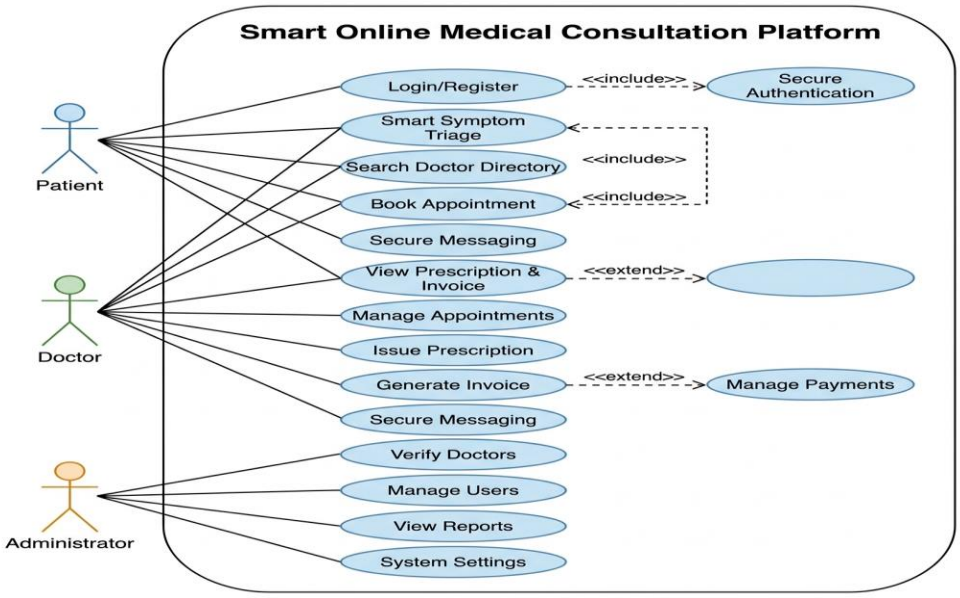


Figure 5.3 — Use Case Diagram: Online Medical Consultation Platform

5.4 Sequence Diagram — AI NLP Symptom Triage

This sequence diagram illustrates the interaction between the Patient, Web Interface, PHP Backend, Python NLP Service, ML Model, and Database during the AI triage process. The patient submits free-text, PHP sanitises and invokes Python via shell_exec, Python tokenises

and classifies the text using the trained model, returns a JSON specialty result, and PHP retrieves matching doctors from the database.

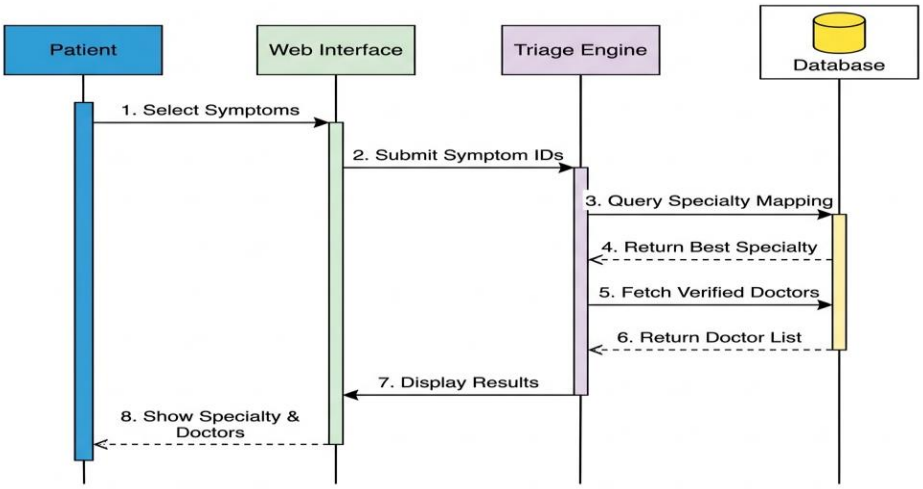


Figure 5.4 — Sequence Diagram: AI NLP Symptom Triage Process

5.5 Sequence Diagram — Appointment Booking

The appointment booking sequence captures the multi-party interaction involved in requesting, validating, and confirming a consultation. It demonstrates the conflict detection logic, the notification sent to the Doctor, and the status update that confirms the appointment, including the conditional branch for a conflict scenario.

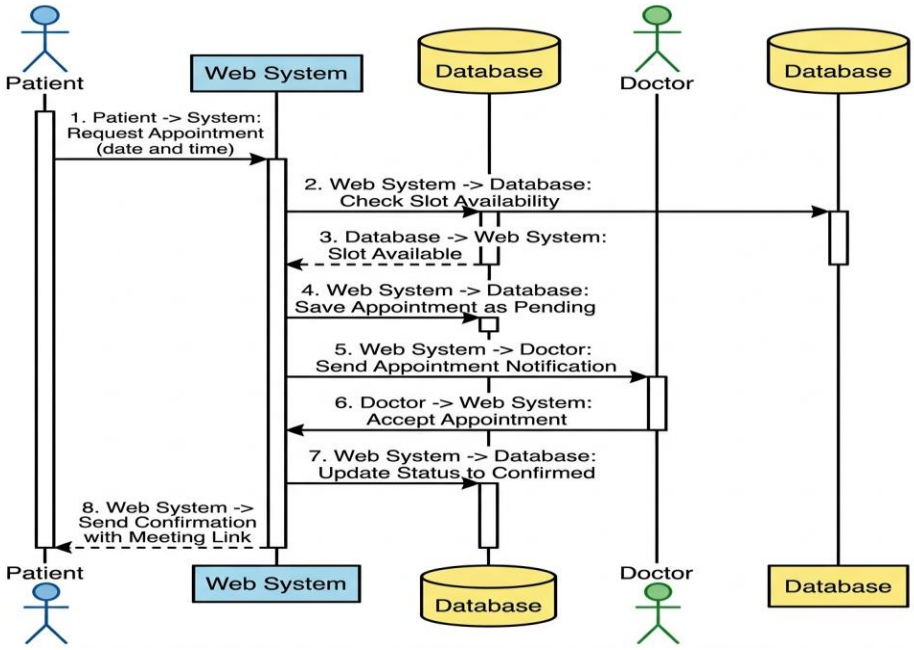


Figure 5.5 — Sequence Diagram: Appointment Booking and Confirmation

5.6 Sequence Diagram — Secure Messaging

This sequence diagram shows the message exchange process between Patient, Chat Interface, Messaging API, and Database. It illustrates both the message submission flow (AJAX POST to store the message) and the periodic retrieval loop (AJAX polling every three seconds to fetch new messages), enabling near-real-time consultation communication.

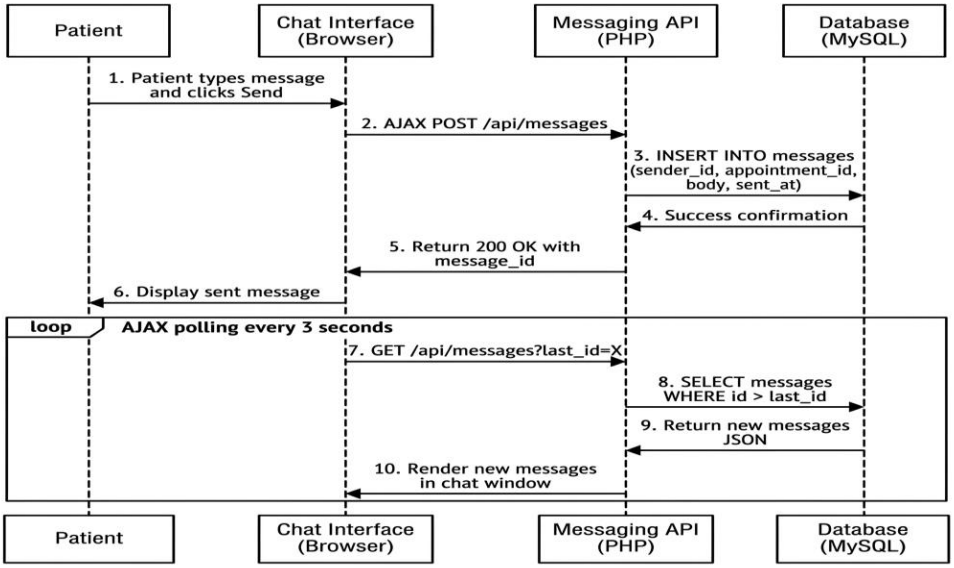


Figure 5.6 — Sequence Diagram: Secure Messaging with AJAX Polling

5.7 Activity Diagram — Consultation Workflow

The Activity Diagram presents the complete end-to-end consultation workflow for both the Patient and the Doctor from initial login through to the conclusion of the consultation. Swimlanes separate the responsibilities of each actor. Decision nodes represent the doctor's acceptance or rejection of an appointment request.

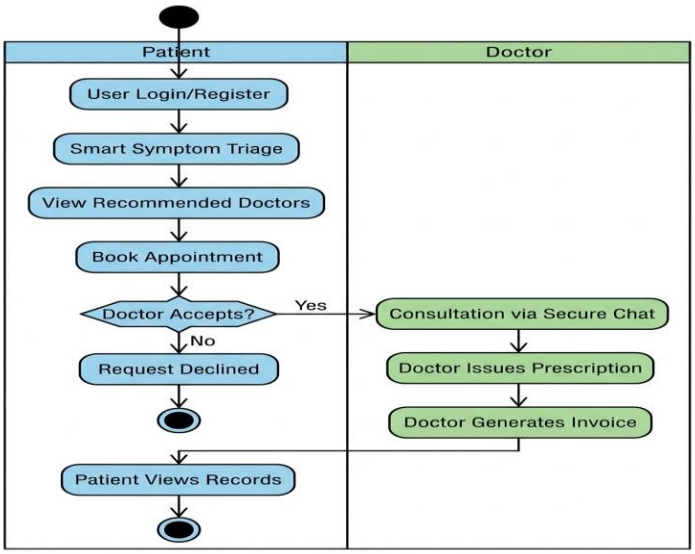


Figure 5.7 — Activity Diagram: End-to-End Consultation Workflow

5.8 Entity Relationship Diagram (ERD)

The ERD defines the database schema showing all primary entities and relationships. The USERS table is the central entity, related to APPOINTMENTS through patient_id and doctor_id foreign keys. APPOINTMENTS link to MESSAGES, PRESCRIPTIONS, INVOICES, and AUDIT_LOG. Crow's foot notation indicates cardinality: one-to-many relationships exist between USERS and APPOINTMENTS, and between APPOINTMENTS and MESSAGES.

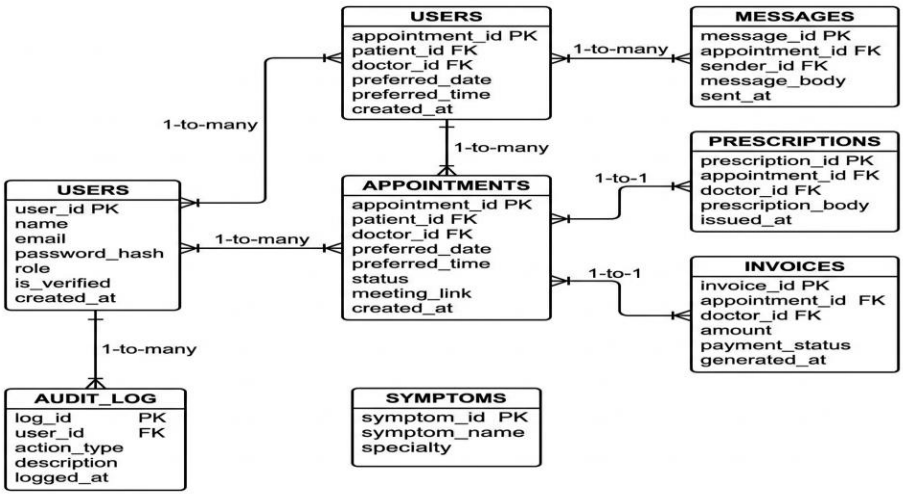


Figure 5.8 — Entity Relationship Diagram: Database Schema

6. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system. They govern how the system performs its functions rather than what functions it performs.

Category	Requirement Detail
Performance	Standard web pages shall load within three to five seconds under normal broadband conditions. The AI NLP triage engine shall process free-text input and return a specialty recommendation within five seconds. The messaging polling cycle shall retrieve and display new messages within three seconds of submission.
Security	User passwords shall be stored using bcrypt hashing. All protected routes shall enforce authentication through server-side session validation. Database queries shall use PDO prepared statements exclusively to prevent SQL injection. Free-text input passed to the Python script shall be escaped using PHP's escapeshellarg() to prevent command injection. All user-supplied input rendered in HTML shall be sanitised to prevent XSS attacks.
Usability	The AI triage input shall require no prior medical knowledge. Navigation elements shall remain consistent across all pages. All form fields shall include descriptive labels and clear error messages. The interface shall be responsive on desktop and tablet screen sizes (min. 1024 × 768 px).
Reliability	The system shall handle AI module failures gracefully, displaying a user-friendly message and directing the patient to the doctor directory. Database connection failures shall present a user-friendly error page. Appointment and prescription records shall not be deletable by patients or doctors.
Scalability	The ML model shall be retrainable by running a single Python script against a new or updated CSV dataset without changes to the PHP application code. The database schema shall support growth to at least 1,000 registered users and 10,000 appointment records without structural modification.
Maintainability	All PHP source files shall follow a consistent naming convention and directory structure. Database schema changes shall be documented. All API endpoints shall return structured JSON responses with consistent status codes.
Availability	The system shall target at least 99% availability during assessed project operation periods. Planned maintenance shall be scheduled outside peak usage hours.

7. Conclusion

The Online Medical Consultation Platform represents a focused and practically achievable contribution to digital healthcare accessibility in Pakistan. By combining an AI-driven NLP Symptom Triage Engine with a structured consultation workflow — encompassing appointment scheduling, secure messaging, digital prescriptions, and invoicing — the system addresses a genuine need: connecting patients with the right medical specialist efficiently, without requiring prior medical knowledge.

The comparative analysis of existing systems confirmed that no currently available platform simultaneously offers AI-based symptom triage, availability in Pakistan, self-hosted deployment capability, and a complete consultation management workflow. The proposed system fulfils all four criteria, establishing both academic and practical significance.

This Software Requirement Specification has defined the full scope of the system with precision. The functional requirements specify every capability the platform must deliver. The non-functional requirements establish the quality standards to be met in terms of performance, security, usability, and reliability. The diagrams in Section 5 provide both a structural and a behavioural model of the system. The appendices supply supporting reference material.

This document provides the complete contractual basis between the project team, the supervisor, and the academic evaluation committee for the subsequent phases of design, implementation, and testing of the Online Medical Consultation Platform.

Appendix A: Glossary

This glossary provides expanded definitions for all technical and domain-specific terms used throughout this document. Terms are listed in alphabetical order.

Term	Full Definition
AJAX	Asynchronous JavaScript and XML. Used in this system for message polling every three seconds without a full page reload.
Apache	An open-source HTTP web server. Serves as the web server layer in the XAMPP stack used by this project.
API	Application Programming Interface. PHP scripts expose JSON endpoints for triage and messaging functions.
Audit Log	A chronological record of system events and user actions, stored in the audit_log database table.
Authentication	Verifying user identity by matching submitted credentials against stored hashed values.
Authorisation	Determining whether an authenticated user has permission to perform a specific action. Implemented via RBAC.
bcrypt	A computationally expensive password hashing algorithm resistant to brute-force attacks. Used for all password storage.
DFD	Data Flow Diagram. A Level 0 DFD (Context Diagram) shows the system as a single process with external entities.
ERD	Entity Relationship Diagram. Shows database entities, attributes, and their relationships.
HTTPS	Hypertext Transfer Protocol Secure. Encrypts all browser-server communication using TLS/SSL.
JSON	JavaScript Object Notation. Used by API endpoints to exchange triage results and message data.
Kaggle	An online data science community providing public datasets. The medical dataset used to train the ML triage model is sourced from Kaggle.
Machine Learning (ML)	A branch of AI where a system learns patterns from data. Used here to predict medical specialties from symptom keywords.
MySQL	An open-source relational database management system used as the data layer of this application.
NLP	Natural Language Processing. Enables computers to understand and process human language. Used here for symptom text analysis.
PDO	PHP Data Objects. Provides parameterised prepared statements to prevent SQL injection.
PHP	Hypertext Preprocessor. The server-side scripting language used to build the application layer.

Python	A high-level programming language used in this project to run the Machine Learning and NLP triage scripts.
RBAC	Role-Based Access Control. This system implements three roles: Patient, Doctor, and Administrator.
scikit-learn	An open-source Python ML library. Used to train the specialty prediction classifier.
Session	A server-side mechanism maintaining state across HTTP requests. PHP sessions track logged-in users and their roles.
shell_exec	A PHP function that executes an OS shell command. Used to invoke the Python NLP script from within PHP.
SQL Injection	A security vulnerability prevented here through the exclusive use of PDO prepared statements.
TF-IDF	Term Frequency-Inverse Document Frequency. An NLP algorithm for identifying important keywords in text.
Telemedicine	The use of telecommunications to provide clinical healthcare services remotely.
Triage	The automated process of mapping patient symptoms to medical specialties to recommend the right practitioner.
UML	Unified Modelling Language. Used in Section 5 to present use case, sequence, activity, and other diagrams.
XAMPP	A free, cross-platform web server stack including Apache, MySQL, and PHP. The deployment environment for this project.
XSS	Cross-Site Scripting. Prevented by sanitising all user-supplied input before rendering it in HTML.

Appendix B: Bibliography and References

The following sources were consulted during the research, analysis, and preparation of this Software Requirement Specification. References are listed in IEEE citation style.

B.1 Standards and Guidelines

- IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," IEEE, New York, USA, 1998.
- ISO/IEC/IEEE 29148:2018, "Systems and software engineering — Life cycle processes — Requirements engineering," ISO, Geneva, 2018.
- IEEE Std 1016-2009, "IEEE Standard for Software Design Descriptions," IEEE, New York, USA, 2009.

B.2 Technical Documentation

- The PHP Group. (2024). PHP Manual. Retrieved from https://www.php.net/manual
- Python Software Foundation. (2024). Python 3 Reference Manual. Retrieved from https://docs.python.org/3/
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Apache Software Foundation. (2024). Apache HTTP Server Documentation.

B.3 Healthcare and Telemedicine References

- World Health Organization. (2023). Pakistan Health Profile 2023. WHO Global Health Observatory. Retrieved from https://www.who.int/countries/pak/
- McKinsey & Company. (2021). Telehealth: A Quarter-Trillion-Dollar Post-COVID-19 Reality. McKinsey Global Institute.

B.4 AI and NLP in Healthcare

- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media, Sebastopol, CA.
- Kaggle.com. (2024). Disease Symptom Prediction Dataset. Retrieved from https://www.kaggle.com

B.5 Software Engineering References

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education, Harlow, England.
- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill, New York.
- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.
- Date, C. J. (2019). An Introduction to Database Systems (8th ed.). Addison-Wesley, Reading, MA.

Appendix C: Document Revision History

This table records all versions and revisions made to this Software Requirement Specification document from initial draft to final submission.

Version	Date	Author(s)	Section(s)	Description of Change
v0.1	March 2026	Project Team	All	Initial draft created based on project scope document and IEEE 830 template.
v0.2	April 2026	Project Team	Sec. 3, 4	Functional requirements expanded; external interface section added.
v0.3	May 2026	Project Team	Sec. 5	UML diagrams added: Use Case, Sequence (Triage, Booking), Activity Diagram.
v0.4	May 2026	Project Team	Sec. 2, 6	Non-functional requirements table added; product functions elaborated.
v1.0	June 2026	Project Team	All	First complete version submitted to supervisor for review.
v1.1	June 2026	Project Team	Sec. 1, 2	Problem Statement, Background, and Objectives sections added per supervisor feedback.
v2.0	June 2026	Project Team	All	Final version incorporating all supervisor revisions. Context Diagram, Architecture Diagram, ERD, and Messaging Sequence Diagram added.
v3.0	June 2026	Project Team	Sec. 2, 5	Replaced UI mockups with operational workflow diagrams (Practo, Zocdoc, Marham). Removed page headers per academic formatting standard.
v6.0	June 2026	Project Team	All	Architecture updated from rule-based triage to AI-driven Python NLP/ML engine trained on medical dataset. Definitions, objectives, and FR-06 to FR-11 updated accordingly.

— End of Document —